



A Machine Learning Approach To Wine Quality Evaluation

CSCI 323

FT14

<https://youtu.be/-RX6cVTDkVg>

<https://github.com/yyan021212/wine-quality-analysis>

Table of content



1. Team Members and Contribution Table

2. Introduction and Background

3. Introduction of Regression Models & Model Setup

4. Regression Models Comparison

5. Introduction of Classification Models & Model Setup

6. Classification Models Comparison

Conclusion

1. Team members and contribution table



Ivan Koh Zhi En, Jin Wan Kim, Renyu Xia, Yuxuan Yan, Xuexin Deng

FT14					
Section	Xuexin Deng	Jin Wan Kim	Ivan Koh Zhi En	Renyu Xia	Yuxuan Yan
1. Introduction			P+S		
2. Research of wine background information			P+S		
3. Introduction of 3 regression models and set up		P+S			
4. Compare and analyses the performance of 3 regression models				P+S	
5. Introduction of 3 classification models and set up					P+S
6. Compare and analyses the performance of 3 classification models	P+S				
7. Conclusion & References			P+S		
Video Production	R+E	R	R	R	R+U



2. Introduction and Background Overview

Introduction



Significance

Consumers:

Wine quality factors; place of origin, price, certification, used to determine value.

Producers:

Quality and certifications used to maintained standards, reduce defects and categorize for sale.

Problem

Sensory evaluation used to score wine quality done by humans is subjective; precision and consistency is limited.

Aim

Evaluate effectiveness of machine learning methods using classification and regression models as an alternative approach to predicting wine quality.

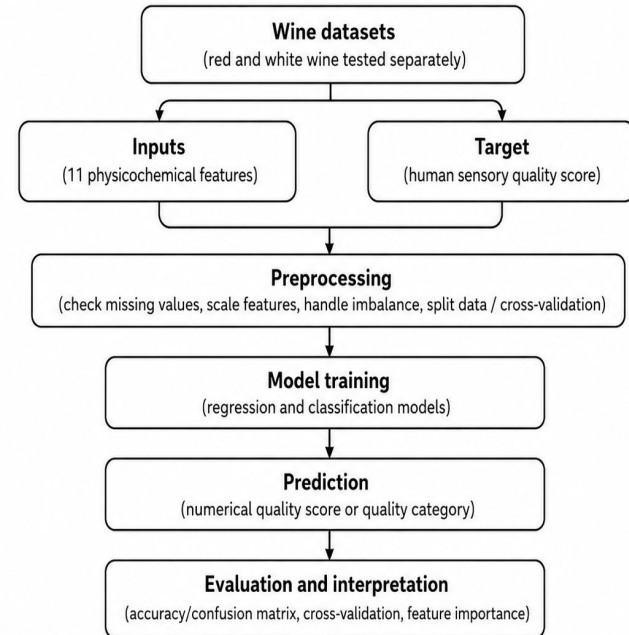


Background Overview



- Sensory evaluations converted to a rating scale of 1-10.
- Classification and regression models use sensory score for prediction.
- Classification categorizes samples into “low”, “medium”, “high” quality, regression outputs numerical prediction.
- Preprocessing done to prepare for model training.

Machine Learning Workflow for Wine Quality





3. Introduction of Regression Models & Model Setup

Linear Regression & SetUp



Linear Regression

What it is:

- Predicts the target variable as a weighted sum of input features
 - Formula: $\hat{y} = w_0 + w_1X_1 + w_2X_2 + \dots + w_{11}X_{11}$
 - Minimises MSE via Ordinary Least Squares (OLS)
 - Assumes linear relationships between features and target
- > may underfit complex patterns

Pipeline + GridSearchCV Setup

- StandardScaler applied -> sensitive to feature scale
- Tuned: fit_intercept [True/False] / CV:5-Fold / Metrics: RMSE,MAE,R²

```
lr_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("linreg", LinearRegression())
])

param_grid_lr = {
    "linreg_fit_intercept": [True, False]
}

lr_grid = GridSearchCV(
    estimator=lr_pipeline,
    param_grid=param_grid_lr,
    cv=5,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)
```

Random Forest Regressor & SetUp



Random Forest Regressor

What it is:

- Ensemble model that builds multiple independent decision trees in parallel
- Averages predictions across all trees -> reduces variance and overfitting
- Captures complex nonlinear interactions without assuming linearity
- No feature scaling required (tree-based model)

GridSearchCV Setup

- Tuned: n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features / CV: 5-Fold / Metrics: RMSE, MAE, R^2

```
rf_reg_param_grid = {
    "n_estimators": [200, 400],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", 0.6, 1.0]
}

rf_reg_grid = GridSearchCV(
    estimator=rf_reg,
    param_grid=rf_reg_param_grid,
    cv=5,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)
```

Gradient Boosting Regressor & SetUp



Gradient Boosting Regressor

What it is:

- Ensemble boosting model that builds decision trees sequentially
- Each new tree learns from the residual errors of the previous tree
- Optimises prediction errors step by step -> captures complex nonlinear patterns
- No feature scaling required (tree-based model)

GridSearchCV Setup

- Tuned: `n_estimators`, `learning_rate`, `max_depth` / CV: 5-Fold / Metrics: RMSE, MAE, R^2

```
gb_param_grid = {
    'n_estimators': [100, 200],
    'learning_rate': [0.05, 0.1],
    'max_depth': [3, 5]
}

gb_grid = GridSearchCV(
    estimator = GradientBoostingRegressor(random_state=42),
    param_grid = gb_param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)
```



4. Regression Models Comparison

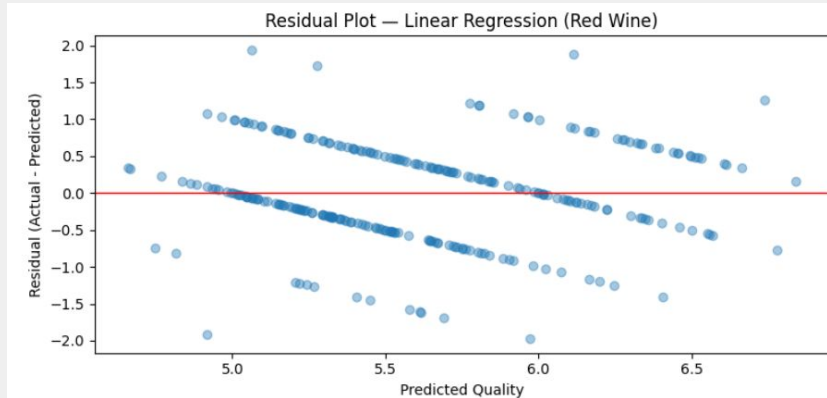
Regression Model Comparison



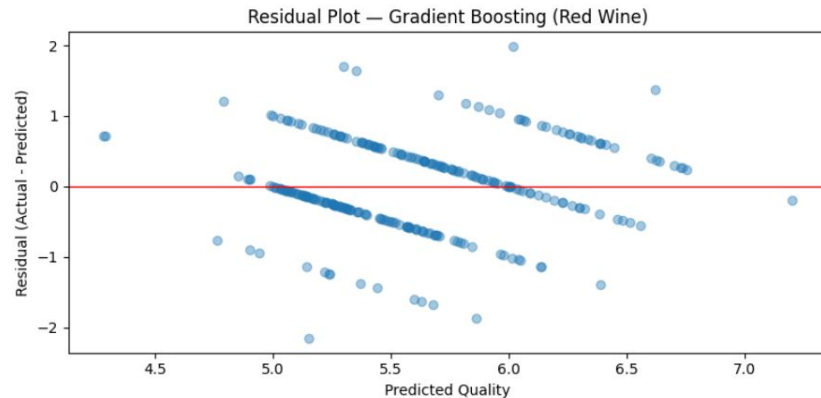
Model	RMSE	MAE	R ²
Linear Regression	0.6629	0.5258	0.3294
Random Forest Regressor	0.6451	0.5057	0.3649
Gradient Boosting Regressor	0.6441	0.5069	0.3670

Gradient Boosting achieved the best overall predictive performance, while Linear Regression showed the weakest performance among the three regression models.

Residual and Overfitting Analysis



Residuals are more spread out around zero
Visible prediction errors remain across different quality ranges
Indicates weaker fitting ability



Residuals are more concentrated around zero
Smaller prediction errors
Better nonlinear fitting capability

Gradient Boosting produced more stable residual distributions, indicating stronger predictive accuracy and better generalisation performance.



5. Introduction of Classification models & Model Setup

Introduction three Classification Models



Gaussian Naive Bayes (GaussianNB)

- Probabilistic classification model
- Assumes features follow Gaussian distribution
- Simple and fast baseline model

Logistic Regression

- Linear classification model
- Predicts probability of each class
- Works well for linearly separable patterns
- Provides interpretable results

Random Forest

- Nonlinear ensemble learning model
- Combines multiple decision trees
- Captures complex feature relationships
- More robust for nonlinear patterns



Why Compare These Models?



Model	Type	Purpose
GaussianNB	Probabilistic	Baseline comparison
Logistic Regression	Linear	Test linear relationships
Random Forest	Nonlinear	Test complex nonlinear relationships

This comparison helps determine whether wine quality prediction is mainly linear or nonlinear.



Pipeline, Cross Validation & Evaluation



Why Use Pipeline?

- Combines preprocessing and model training
- Data is scaled before classification

Why RobustScaler?

- Wine data contains outliers
- Reduces influence of extreme values
- Improves Logistic Regression stability

Why `class_weight="balanced"`?

- Wine quality classes are imbalanced
- Prevents bias toward majority classes

Logistic Regression Pipeline

```
Pipeline([  
    ("scaler", RobustScaler()),  
    ("clf", LogisticRegression(  
        max_iter=4000,  
        solver="lbfgs",  
        class_weight="balanced",  
        random_state=42  
    ))  
])
```



Pipeline, Cross Validation & Evaluation



Why Use Cross Validation?

- More reliable than one train-test split
- Better estimate of generalisation performance
- Stratification preserves class distribution

Why Use Macro F1?

- Balances precision and recall
- Treats all classes equally
- Suitable for imbalanced datasets

Stratified 5-Fold Cross Validation

```
cv = StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=42  
)
```

Evaluation Metrics

```
scoring = {  
    "acc": "accuracy",  
    "f1_macro": "f1_macro",  
    "precision_macro": "precision_macro",  
    "recall_macro": "recall_macro"  
}
```





6. Classification Models Comparison and Analysis

Best Classification Model Selection



White Wine

	Model	CV_Acc_mean	CV_Acc_std	CV_F1macro_mean	CV_F1macro_std	CV_Prec_macro_mean	CV_Rec_macro_mean
2	RandomForest	0.607954	0.020130	0.590279	0.021462	0.623858	0.576667
1	LogisticRegression	0.546720	0.024919	0.546316	0.024923	0.545600	0.588103
0	GaussianNB	0.502208	0.027355	0.507014	0.027091	0.508757	0.549621

Red Wine

	Model	CV_Acc_mean	CV_Acc_std	CV_F1macro_mean	CV_F1macro_std	CV_Prec_macro_mean	CV_Rec_macro_mean
2	RandomForest	0.672532	0.028254	0.618453	0.037074	0.670216	0.599814
1	LogisticRegression	0.602575	0.023788	0.570616	0.019916	0.566026	0.623709
0	GaussianNB	0.601700	0.027820	0.562348	0.037082	0.571902	0.562189



Fine-tuning for Random Forest



GridSearch

```
param_grid = {  
    "n_estimators": [200, 400],  
    "max_depth": [None, 10, 15, 20],  
    "min_samples_split": [2, 5, 10],  
    "min_samples_leaf": [1, 2, 4],  
    "max_features": ["sqrt", 0.6, 1.0]
```

Red values represent the best parameters

f1_macro before GridSearch: 0.618

f1_macro after GridSearch: **0.623**

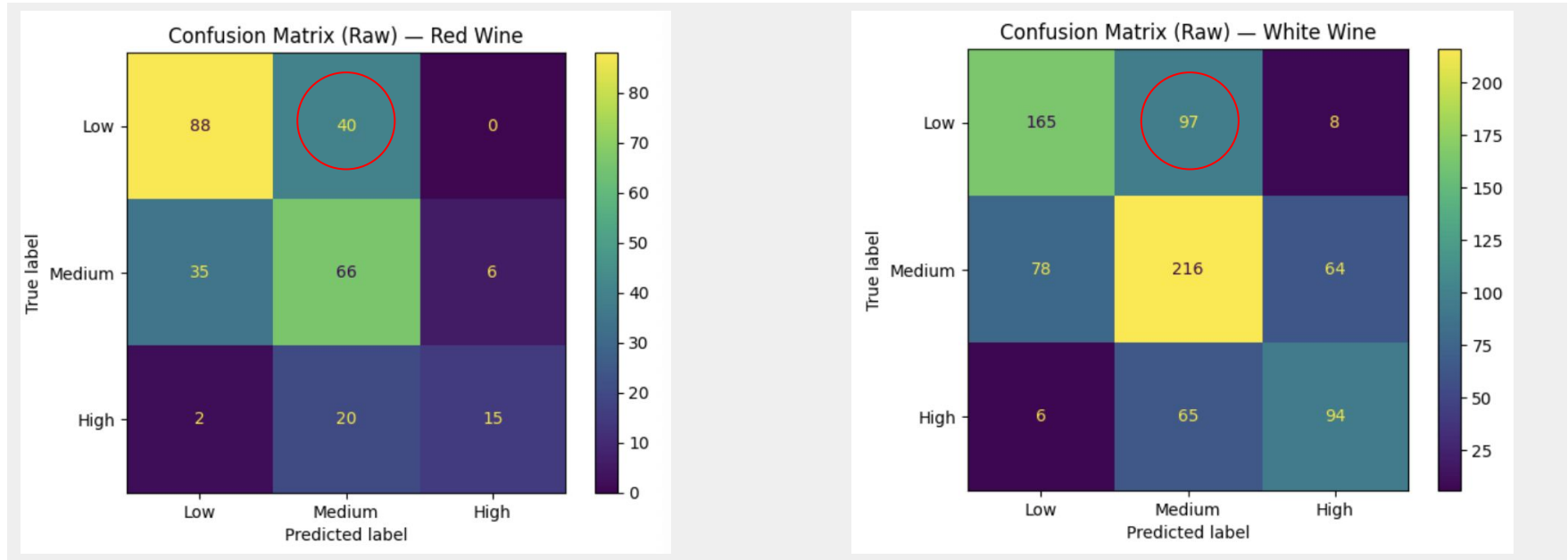
Overfitting Analysis

	F1_Macro	Accuracy
Train	0.9892	0.9899
Test	0.5931	0.6213
Gap	0.3961	0.3686

Red Wine Database as example



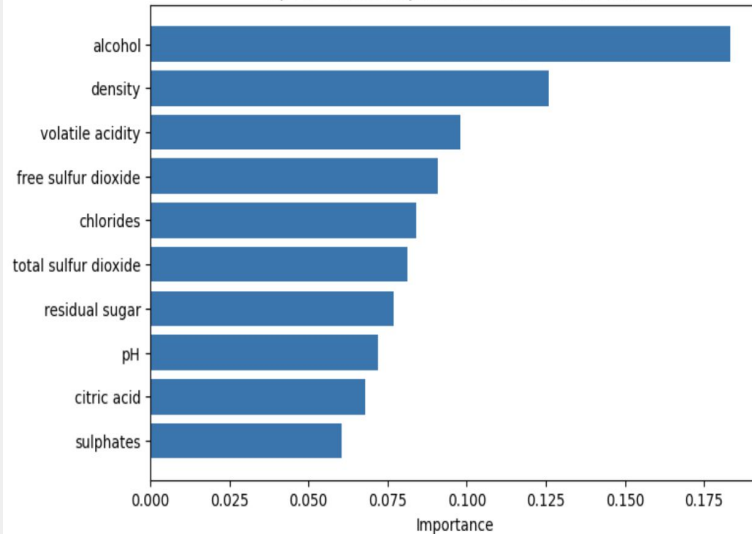
Confusion Matrix for Random Forest



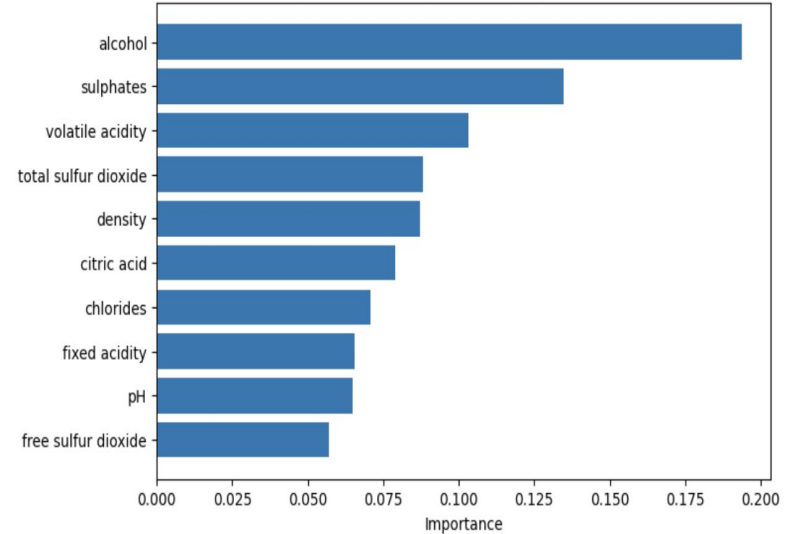
Feature Important



Top 10 Feature Importances (RF) — White Wine



Top 10 Feature Importances (RF) — Red Wine



Conclusion



Observations

- Physicochemical lab measurements contain useful information to predict sensory score.
- However, correlation is moderate; insufficient to replace sensory based evaluations.
- Machine learning approach suitable as secondary assessment in addition to human sensory evaluation.

Improvements for future work

- More advanced models such as support vector regression or XGBoost could be explored.
- Use dataset from different source to assess if results can be generalized to outside of initial dataset.

Overall Findings

Ensemble regression models demonstrated stronger capability in modelling complex physicochemical relationships for wine quality prediction.



References



- Australian Wine Research Institute. (n.d.). Practical sensory evaluation considerations.
- Australian Wine Research Institute. (n.d.). Scope of operation.
- UCI Machine Learning Repository. (2009). Wine Quality.

